

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COMPUTER VISION

ITA0503

VISIONGUARD

Multi-Module Vision System Design: Traditional vs. Deep-Learning Approach for Campus Access and Surveillance

ASSIGNMENT REPORT

Student Name	B.Chaithanya Prakash reddy R.Tej Deep reddy K.Anil kumar reddy
Register / Roll No.	192425255 192425154 192525239
Section / Slot	C
Course Code	ITA0503
Academic Year	2026

Submitted in partial fulfilment of the requirements for the Computer Vision course.

DECLARATION

I hereby declare that this report titled “VisionGuard – Multi-Module Vision System Design: Traditional vs. Deep-Learning Approach for Campus Access and Surveillance” has been prepared for the ITA05 Computer Vision assignment. The report follows the problem statement, learning outcomes, and deliverables specified in the assignment brief. The design, analysis, pseudocode, and proposed implementation are presented for academic evaluation.

TABLE OF CONTENTS

1. Executive Summary
2. Problem Understanding and Vision-System Requirements
3. Objectives and Scope
4. System Architecture
5. Image Acquisition and Preprocessing
6. Module 1 – Face Recognition
7. Module 2 – Vehicle Number Plate Recognition
8. Module 3 – Library Surveillance and Unusual-Activity Detection
9. Traditional Computer Vision vs. Deep Learning
10. Selected Approach and Justification
11. End-to-End Algorithm and Pseudocode
12. Implementation Plan
13. Dataset, Training, Validation and Testing
14. Evaluation Metrics and Result Presentation
15. Deployment, Privacy and Ethical Considerations
16. SDG and Industrial Relevance
17. Limitations and Future Enhancements
18. Conclusion
19. References / Tools

1. EXECUTIVE SUMMARY

VisionGuard is an integrated OpenCV-based campus vision system designed around three practical security functions: recognizing registered students or staff at an access gate, reading vehicle number plates at the parking entrance, and identifying unusual activity in a library surveillance feed. The assignment brief emphasizes that the system must remain reliable under lighting changes, camera-angle variation, and image noise, while balancing accuracy, adaptability, computational requirements, training requirements, deployment constraints, and privacy.

The proposed design uses a hybrid strategy rather than forcing one technique across all modules. Image enhancement and segmentation are used before recognition or detection. Traditional methods such as Haar cascades, LBPH, template matching, and background subtraction remain useful where the environment is controlled and computational resources are limited. Deep-learning methods such as CNN-based face embeddings, deep OCR, YOLO-style object detection, and learned anomaly detection are preferred where appearance variation and robustness are more important.

The recommended system therefore uses deep learning as the primary recognition/detection layer and OpenCV as the common image-processing and integration layer. This provides a practical balance: preprocessing can improve image quality, learned models can handle difficult visual variation, and the overall pipeline can still be deployed as a modular application. Privacy controls, access restrictions, retention limits, and responsible use of biometric and vehicle information are treated as deployment requirements rather than optional additions.

Key outcome

- The system is divided into face access, plate recognition, and surveillance modules.
- Preprocessing includes resizing, denoising, illumination correction, contrast enhancement, and segmentation/region-of-interest extraction.
- Traditional methods are lightweight and interpretable but generally less adaptive to pose, illumination, and appearance changes.
- Deep-learning methods offer stronger adaptability and accuracy potential but require training data, validation, model management, and greater computation.
- A hybrid OpenCV-centered architecture is recommended, with deep models used for the tasks where robustness is most critical.

2. PROBLEM UNDERSTANDING AND VISION-SYSTEM REQUIREMENTS

The assignment brief defines VisionGuard as a campus security and monitoring system serving three locations: the main entrance, parking entrance, and library. Its functional requirement is not simply to detect objects; it must convert camera data into useful security decisions. The face module must recognize registered people, the parking module must read vehicle plates, and the surveillance module must flag unusual activity. The system must operate despite common CCTV imperfections.

2.1 Functional requirements

Module	Input	Processing	Output
Access Gate	Face image/video frame	Face detection, alignment, feature extraction, identity matching	Registered identity / unknown / access decision
Parking Entrance	Vehicle image/video frame	Vehicle/plate localization, plate enhancement, character recognition	Detected plate text and confidence
Library Surveillance	Continuous video	Motion/object detection, tracking and anomaly rules/model	Normal activity / unusual activity alert

2.2 Non-functional requirements

- Reliability: reduce false acceptance, false recognition, missed plates, and unnecessary surveillance alerts.
- Robustness: tolerate illumination variation, blur, moderate noise, pose changes, and camera-angle variation.
- Performance: process frames with acceptable latency for the intended campus use case.
- Scalability: allow additional cameras, registered identities, vehicles, and monitoring zones.
- Maintainability: keep face, plate, and surveillance components modular so that one model can be updated without redesigning the whole system.
- Security and privacy: restrict access to biometric and vehicle data and retain only what is necessary.

2.3 Assignment alignment

Course Outcome	How the report addresses it
CO2	Image enhancement, filtering, segmentation, edge/feature extraction, and region-of-interest processing.
CO3	Object detection, recognition, OCR, pattern recognition, and deep-learning alternatives.
CO4	Accuracy, adaptability, computation, training effort, robustness, and deployment trade-off analysis.

CO5	End-to-end OpenCV-centered architecture, pseudocode, implementation plan, testing, and visualization.
-----	---

3. OBJECTIVES AND SCOPE

3.1 Objectives

1. Acquire and preprocess camera images or video frames using OpenCV operations.
2. Detect relevant regions such as faces, vehicles, number plates, and moving objects.
3. Compare traditional computer vision methods with deep-learning alternatives for each module.
4. Select the most suitable approach for each module using performance-versus-adaptability reasoning.
5. Design an integrated pipeline that can be implemented and tested as separate modules.
6. Define measurable evaluation metrics and visual outputs for model validation.
7. Address privacy, ethical, and deployment constraints associated with biometric and vehicle data.

3.2 Scope

The scope covers the computer-vision processing chain from camera input to detection, recognition/OCR, alert generation, and result visualization. The system is designed for a campus environment rather than unrestricted public surveillance. It assumes that cameras are positioned to provide usable views and that authorized reference data are available for registered people and permitted vehicle records.

3.3 Out of scope

- Physical gate hardware control, turnstiles, and access-card electronics.
- Legal compliance certification for a particular jurisdiction.
- Guaranteed recognition under severe occlusion or extremely poor image quality.
- Replacing human security personnel for high-impact decisions.
- Automatic disciplinary action based only on a model prediction.

SYSTEM ARCHITECTURE

VisionGuard follows a modular pipeline. Each camera stream first enters a common acquisition and preprocessing layer. The processed frames are then routed to the module appropriate to the camera location. Results are normalized into structured events and displayed through a monitoring interface.

4.1 High-level pipeline

Camera Input → Frame Capture → Quality Check → Preprocessing → Region-of-Interest Detection → Module Model/Algorithm → Confidence Check → Decision/Alert → Logging & Visualization

4.2 Architecture components

Layer	Main responsibility	Example techniques
Acquisition	Read image/video frames	OpenCV VideoCapture, image loading
Preprocessing	Improve quality and standardize input	Resize, Gaussian/median filtering, CLAHE, grayscale, normalization
Segmentation / ROI	Isolate useful visual regions	Thresholding, contours, morphology, crop regions
Detection	Locate objects/regions	Haar, contour methods, YOLO-type detector
Recognition	Identify people or read text	LBPH, embeddings, OCR
Temporal analysis	Understand motion/activity	Frame differencing, background subtraction, tracking
Decision	Convert scores into actions	Thresholds, allowlists, anomaly rules
Presentation	Show results to operator	Bounding boxes, labels, confidence, alerts, logs

4.3 Data flow by module

- Face: frame → face ROI → alignment/normalization → feature representation → identity comparison → access result.
- Plate: frame → vehicle ROI → plate ROI → denoising/contrast/thresholding → character recognition → plate string.
- Surveillance: video → background/object analysis → motion/object tracks → activity features → anomaly decision → alert.

5. IMAGE ACQUISITION AND PREPROCESSING

Preprocessing is important because recognition performance depends on the quality and consistency of the visual input. The assignment specifically requires enhancement, filtering, segmentation, and feature extraction. The proposed pipeline applies these operations selectively rather than applying every operation to every frame.

5.1 Noise removal

Median filtering is suitable for impulse-like noise, while Gaussian filtering provides smoothing for small variations. The filter should be selected carefully because excessive smoothing can remove edges and character details needed by plate OCR or face detection.

5.2 Illumination and contrast correction

Brightness changes are common in campus CCTV footage. Histogram equalization or CLAHE can improve local contrast. Gamma correction may also be used when images are consistently too dark or too bright. These operations should be validated because aggressive enhancement can amplify noise.

5.3 Segmentation and morphology

Thresholding can separate foreground regions from background when the visual conditions are suitable. Morphological opening can remove small noise and closing can connect broken regions. Contours can then be extracted to identify candidate regions such as rectangular number plates.

5.4 Feature extraction

- Edges: useful for boundaries, plate rectangles, and structural information.
- Corners/keypoints: useful for local feature matching where classical methods are selected.
- Face embeddings: learned feature vectors that represent identity-related facial characteristics.
- Character patterns: shape/texture information used by OCR or template matching.

5.5 Preprocessing pseudocode

BEGIN

1. Read frame from camera or video.
2. Resize frame to the model's expected resolution.
3. Convert to an appropriate color space or grayscale representation.
4. Apply noise reduction when image quality requires it.
5. Apply illumination/contrast correction.
6. Detect or define the region of interest.
7. Apply segmentation/morphological operations when required.
8. Pass the cleaned ROI to the selected detection/recognition stage.

END

6. MODULE 1 – FACE RECOGNITION

The access-gate module identifies registered students or staff. A complete system should separate face detection from identity recognition. Detection answers where a face is present; recognition answers whose face it is.

6.1 Traditional approach

A classical pipeline can use a Haar cascade for face detection followed by LBPH face recognition. The method is lightweight and relatively easy to interpret. It can work well in a controlled gate where camera position, lighting, and enrollment conditions are stable. Its weakness is reduced robustness when pose, illumination, expression, occlusion, or appearance changes substantially.

6.2 Deep-learning approach

A deep-learning pipeline can detect the face, align it, and convert it into a learned embedding. The embedding is compared with enrolled reference embeddings using a similarity measure. This approach

generally provides better adaptability to real-world variation, but it requires a suitable pretrained model or training data, validation, and more computation.

6.3 Recommended design

Deep-learning face embeddings are recommended for the primary recognition layer, with OpenCV used for image acquisition, preprocessing, ROI handling, and visualization. For a controlled low-resource deployment, LBPH can be retained as a baseline or fallback for comparative evaluation.

6.4 Face recognition pseudocode

1. Capture frame.
2. Detect all candidate faces.
3. For each face, crop and align the ROI.
4. Normalize the face image.
5. Generate a face feature representation.
6. Compare the representation against registered reference representations.
7. If similarity is above the validated threshold, assign the identity; otherwise label it unknown.
8. Log only the minimum information needed for the access decision.

7. MODULE 2 – VEHICLE NUMBER PLATE RECOGNITION

The parking module converts a vehicle image into readable plate text. It requires both localization and recognition. A plate that is not correctly cropped will often produce incorrect OCR even if the OCR model itself is accurate.

7.1 Traditional approach

A traditional pipeline can use grayscale conversion, denoising, edge detection, thresholding, contour extraction, and geometric filtering to locate a plate. Characters can then be recognized using template matching or other feature-based techniques. This is fast and interpretable but sensitive to font variation, plate style, skew, glare, blur, and low contrast.

7.2 Deep-learning approach

A learned detector can localize vehicles and/or plates, while a deep OCR model recognizes the characters. This is more adaptable to variation in plate appearance and image conditions, but it increases model and computation requirements.

7.3 Recommended design

A detector-plus-OCR pipeline is recommended. OpenCV should handle frame acquisition, image enhancement, plate ROI extraction, perspective correction, and visualization. A learned OCR component should perform character recognition where reliable text extraction is required.

7.4 Plate processing sequence

8. Detect the vehicle or plate region.
9. Crop the plate ROI and correct perspective if necessary.
10. Convert to grayscale and reduce noise.
11. Enhance local contrast.
12. Use adaptive thresholding or another segmentation method when useful.
13. Run OCR on the cleaned plate image.
14. Validate the recognized string using the expected plate-format rules.
15. Store the plate result with a confidence score and timestamp according to the retention policy.

8. MODULE 3 – LIBRARY SURVEILLANCE AND UNUSUAL-ACTIVITY DETECTION

The library module differs from the first two modules because it is temporal. The system must interpret changes across consecutive frames instead of analyzing only one image.

8.1 Traditional approach

Background subtraction and frame differencing can identify regions of motion. Morphological operations can clean the resulting foreground mask, and contours can be used to estimate moving regions. This is computationally efficient and useful in a relatively fixed camera scene. However, it can generate false alarms from shadows, lighting changes, camera vibration, or other background changes.

8.2 Deep-learning approach

A detector such as YOLO can identify relevant objects, while tracking can connect detections over time. A learned anomaly/activity model can then be used where the definition of unusual behavior requires more than simple motion. Deep methods are more adaptable but require suitable data and careful definition of what constitutes an anomaly.

8.3 Recommended design

Use a hybrid approach. Start with efficient motion analysis for continuous screening, then apply object detection and/or learned activity analysis to candidate events. This reduces unnecessary deep-model computation while improving the quality of alerts.

8.4 Example event logic

- Motion detected in a restricted zone → inspect the region.
- Person/object detected → track across frames.
- Event duration, location, direction, and object class are evaluated.
- If a validated unusual-activity condition is met → generate an alert.
- Human review is recommended before high-impact action.

9. TRADITIONAL COMPUTER VISION VS. DEEP LEARNING

Factor	Traditional CV	Deep Learning
Accuracy potential	Good in controlled conditions	Generally higher potential in varied real-world conditions
Adaptability	Limited when appearance/lighting changes	Strong if trained data represent deployment conditions
Computation	Low to moderate	Moderate to high, depending on model
Training	Little or none for many classical methods	Requires training or suitable pretrained models
Interpretability	High; individual operations are easy to inspect	Lower; learned representations are less directly interpretable
Data requirement	Lower	Higher, especially for custom training
Deployment	Simple on low-resource hardware	May need GPU/accelerator or optimized inference
Maintenance	Manual rule/threshold tuning	Model updates and monitoring required
Robustness	Sensitive to pose, illumination, clutter	Better tolerance when training covers variation
Best use	Controlled environments and lightweight baselines	Complex recognition/detection under varied conditions

9.1 Module-wise comparison

Module	Traditional candidate	Deep-learning candidate	Preferred
Face recognition	Haar + LBPH	CNN face embeddings	Deep learning
Plate reading	Contours + template matching	Detector + deep OCR	Deep learning
Surveillance	Background subtraction + rules	YOLO/object detection + learned activity/anomaly model	Hybrid

9.2 Trade-off interpretation

There is no single universally best method. Traditional methods are attractive when the camera environment is controlled, the hardware is limited, and interpretability is a priority. Deep learning becomes more valuable when the system must handle variation in pose, lighting, scale, clutter, and appearance. The recommended design therefore assigns methods according to the actual difficulty of each module.

10. SELECTED APPROACH AND JUSTIFICATION

10.1 Face module – deep learning

Identity recognition is sensitive to facial appearance and imaging conditions. A learned embedding is therefore selected because it can represent identity using features learned from data rather than relying

only on handcrafted image patterns. The final access decision must still use a validated similarity threshold and an unknown-person path.

10.2 Plate module – deep learning with OpenCV preprocessing

Plate reading combines localization and character recognition. Deep detection and OCR are preferred because plate images can vary in size, skew, font rendering, lighting, and background. OpenCV preprocessing remains valuable for improving the OCR input and for making the pipeline easier to inspect.

10.3 Surveillance – hybrid

Continuous video processing can be computationally expensive. A motion-based first stage can identify candidate changes efficiently, after which a learned detector or activity model can analyze only relevant frames/regions. This provides a practical balance between speed and adaptability.

10.4 Decision matrix

Criterion	Face	Plate	Surveillance
Accuracy priority	Very high	Very high	High
Adaptability need	High	High	High
Traditional method role	Baseline/fallback	ROI preprocessing/baseline	Fast screening
Deep-learning role	Primary	Primary	Secondary/primary event analysis
Recommended strategy	Deep	Deep + OpenCV	Hybrid

11. END-TO-END ALGORITHM AND PSEUDOCODE

11.1 Main system pseudocode

BEGIN VISIONGUARD

Load configuration, registered identities, permitted vehicle information, and trained/pretrained models.

Open camera streams for gate, parking, and library.

WHILE cameras are active:

 Capture current frame.

 Perform quality check and preprocessing.

 IF source is gate:

 Detect face(s).

 Align and normalize face ROI.

 Generate face representation.

 Compare with registered identities.

Generate access result.

ELSE IF source is parking:

Detect vehicle/plate.

Crop and enhance plate ROI.

Run OCR.

Validate recognized plate string.

ELSE IF source is library:

Perform motion/background or object analysis.

Track relevant activity.

Evaluate anomaly/activity conditions.

Generate alert if validated.

Display bounding boxes, labels, confidence, and status.

Store only required event information.

END WHILE

Release cameras and resources.

END VISIONGUARD

11.2 Feature-processing pseudocode

Input image → resize → denoise → illumination correction → grayscale/color conversion → ROI detection → segmentation → morphological cleanup → edge/keypoint extraction where required → model/recognition stage.

11.3 Confidence handling

- Use separate thresholds for detection confidence and recognition similarity.
- Reject or route low-confidence results for human review rather than forcing an identity.
- Record confidence values during testing so thresholds can be tuned using validation data.
- Evaluate false positives and false negatives, not only overall accuracy.

12. IMPLEMENTATION PLAN

The implementation can be organized as independent Python modules so that each component can be tested separately before integration.

Component	Suggested implementation responsibility
capture.py	Camera/video input and frame management
preprocess.py	Resize, filtering, enhancement, grayscale/normalization
face_module.py	Face detection, embedding/recognition, access decision
plate_module.py	Vehicle/plate detection, ROI correction, OCR, format validation
surveillance_module.py	Motion detection, object detection, tracking, anomaly rules
evaluation.py	Metrics, confusion matrix, timing, threshold analysis
visualization.py	Bounding boxes, labels, alerts, result images
main.py	Pipeline integration and configuration

12.1 Software flow

16. Install Python and required computer-vision libraries.
17. Create separate folders for face, plate, and surveillance datasets.
18. Implement preprocessing and test it independently.
19. Implement and validate each module on its own.
20. Measure accuracy and inference time before integration.
21. Integrate the three modules through a common frame/event interface.
22. Run end-to-end tests using representative camera conditions.
23. Document installation, execution, results, and limitations in the project README.

12.2 Deployment considerations

- Use hardware appropriate to the selected models and camera resolution.
- Prefer smaller or optimized models when real-time latency is more important than marginal accuracy.
- Maintain model/version information for reproducibility.
- Monitor camera health and image quality because model performance depends on input quality.
- Separate operator access from administrator access.

13. DATASET, TRAINING, VALIDATION AND TESTING

The assignment requires dataset information for face images, plate images, and a surveillance clip. Exact dataset sizes, sources, and measured results should be filled in from the actual implementation records. They are intentionally not invented in this report.

Dataset	Required content	Recommended split/handling
Face dataset	Multiple images per registered identity with variation in pose and lighting	Training/enrollment and held-out validation/test images
Plate dataset	Vehicle/plate images covering different angles, illumination, and plate appearances	Training/validation/test; preserve unseen scenes for final testing
Surveillance clip	Normal and representative unusual-activity sequences	Separate development clips from final evaluation clips

13.1 Training principles

- Keep test data isolated from training and threshold tuning.
- Include representative variation in lighting, camera angle, blur, scale, and background.
- Use augmentation when appropriate to improve robustness.
- Record model configuration, preprocessing settings, and threshold values.
- Evaluate on data that reflect the actual campus deployment conditions.

13.2 Testing scenarios

Scenario	Purpose
Daylight	Baseline performance
Low light	Test illumination robustness
Backlight	Test contrast and dynamic-range limitations
Side/angled face	Test pose variation
Blur/noise	Test CCTV image degradation
Partial occlusion	Test robustness to masks/objects
Crowded scene	Test multiple-object detection
Plate glare/skew	Test OCR robustness
Background movement	Test surveillance false alarms

14. EVALUATION METRICS AND RESULT PRESENTATION

The report should present measured results from the actual implementation. Accuracy alone is insufficient because the system has different failure costs across modules.

14.1 Face recognition metrics

- Recognition accuracy on the held-out test set.
- False acceptance rate and false rejection rate where access-control testing is available.
- Unknown-person rejection performance.
- Average inference time per face/frame.

14.2 Plate recognition metrics

- Plate detection precision and recall.
- Character-level OCR accuracy.
- Complete-plate accuracy: percentage of images for which the entire plate string is correct.

- Average processing time per vehicle/image.

14.3 Surveillance metrics

- Detection precision and recall for relevant objects/events.
- False-alarm rate.
- Missed-event rate.
- Alert latency.
- Frames per second or average processing time.

14.4 Results table template

Module	Metric	Traditional	Deep/Hybrid	Selected
Face	Recognition accuracy	To be measured	To be measured	Deep
Face	Inference time	To be measured	To be measured	Deep
Plate	Complete-plate accuracy	To be measured	To be measured	Deep
Plate	OCR character accuracy	To be measured	To be measured	Deep
Surveillance	False-alarm rate	To be measured	To be measured	Hybrid
Surveillance	Alert latency	To be measured	To be measured	Hybrid

14.5 Visualizations to include in final implementation

- Sample face detection and recognition frames with bounding boxes and identity labels.
- Plate detection images showing the original frame, cropped plate, and OCR output.
- Surveillance frames showing detected objects/motion and an example alert.
- Confusion matrix or class-wise precision/recall where applicable.
- Training/validation curves for custom deep-learning models.
- Comparison chart of accuracy versus inference time for selected alternatives.

15. DEPLOYMENT, PRIVACY AND ETHICAL CONSIDERATIONS

VisionGuard handles information that can be sensitive in a campus setting. Face recognition involves biometric information and plate recognition can associate vehicles with individuals. Therefore, technical performance must be accompanied by responsible data handling.

15.1 Privacy controls

- Collect only data necessary for the stated campus-security purpose.
- Restrict access to enrolled face data and vehicle records.
- Use secure storage and controlled credentials.
- Define retention periods and delete data when no longer required.
- Separate raw video from event metadata where practical.
- Keep an audit trail for administrative access to sensitive records.

15.2 Ethical safeguards

- Do not treat a low-confidence model prediction as unquestionable truth.
- Provide an unknown/uncertain state rather than forcing recognition.
- Use human review for consequential access or disciplinary decisions.
- Evaluate whether performance differs across relevant operating conditions.
- Clearly define authorized monitoring zones and purposes.
- Inform users about the existence and purpose of the system according to institutional policy.

15.3 Reliability principle

A vision system should assist campus security personnel rather than remove human accountability. The most important design goal is not simply to maximize model accuracy but to create a system whose errors are visible, reviewable, and manageable.

16. SDG AND INDUSTRIAL RELEVANCE

SDG	Relevance to VisionGuard
SDG 9 – Industry, Innovation and Infrastructure	Demonstrates application of computer vision, AI, and automated monitoring infrastructure in a real-world institutional environment.
SDG 11 – Sustainable Cities and Communities	Supports safer and more organized campus spaces through automated monitoring and access management.
SDG 16 – Peace, Justice and Strong Institutions	Promotes controlled access, accountable monitoring, and responsible security processes.

Industrial relevance

The same architecture can be adapted to office campuses, educational institutions, industrial facilities, parking systems, and controlled-access environments. The main industrial lesson is that model selection must follow the operating environment. A lightweight classical solution may be appropriate for a fixed camera and simple scene, while a learned detector or recognition model is more suitable when the environment is variable.

Learning outcomes achieved

- Applied image enhancement, filtering, segmentation, and feature analysis.
- Analyzed the strengths and weaknesses of classical computer vision.
- Evaluated deep-learning models in terms of adaptability and computational demand.
- Designed a modular computer-vision system using OpenCV as the integration layer.
- Considered privacy and ethical constraints in a real-world deployment.

17. LIMITATIONS AND FUTURE ENHANCEMENTS

17.1 Current limitations

- Performance depends strongly on camera placement and image quality.
- Traditional approaches may degrade under uncontrolled lighting and pose changes.
- Deep models require suitable data and computational resources.
- Surveillance anomalies are difficult to define because unusual behavior is context dependent.
- Thresholds require validation; a single universal confidence value is not reliable for every deployment.
- Privacy requirements can limit the amount of data that may be retained for training or auditing.

17.2 Future enhancements

- Add multi-camera tracking and event correlation.
- Use model optimization or hardware acceleration for real-time deployment.
- Introduce better plate perspective correction and OCR confidence fusion.
- Add temporal activity recognition for more context-aware surveillance.
- Implement secure model and data versioning.
- Create a dashboard for camera status, alerts, and performance monitoring.
- Continuously evaluate the system using newly observed but appropriately governed deployment conditions.

18. CONCLUSION

VisionGuard provides a practical framework for applying computer vision to three campus-security tasks: face recognition, vehicle plate reading, and library surveillance. The analysis shows that traditional computer vision remains valuable because it is fast, lightweight, and interpretable, but its performance can decline when real-world conditions differ from the controlled assumptions of handcrafted methods.

Deep learning is therefore selected as the primary approach for face recognition and plate reading, while surveillance is designed as a hybrid pipeline combining efficient motion analysis with learned object or activity understanding. OpenCV serves as the common foundation for acquisition, enhancement, segmentation, ROI processing, visualization, and system integration.

The final design emphasizes that successful computer-vision deployment is not determined by model accuracy alone. Accuracy must be balanced with adaptability, latency, hardware requirements, training effort, maintainability, privacy, and the consequences of errors. With appropriate validation and responsible data handling, VisionGuard can improve the efficiency of campus access control and surveillance while keeping human oversight in the decision loop.

19. REFERENCES / TOOLS

The assignment brief provided with this report is the primary source for the problem statement, course outcomes, assessment criteria, SDG mapping, and deliverables. The following software/tool categories are appropriate for implementation:

- Python – application development and experimentation.
- OpenCV – image acquisition, preprocessing, filtering, segmentation, video processing, and visualization.
- NumPy – numerical image-array operations.
- Deep-learning framework – model training/inference where required.
- YOLO-family detector or equivalent object detector – vehicle/object detection where selected.
- OCR engine/model – number-plate character recognition.
- Git/GitHub – source-code versioning and project documentation.

Assignment brief alignment

The source assignment identifies the course as ITA05 Computer Vision and requires problem analysis/pseudocode, implementation and results, and a GitHub upload containing source code, dataset information, README, execution instructions, evaluation, and documentation.

□filecite□turn0file0□L32-L36□ □filecite□turn0file0□L142-L145□

It also specifies the VisionGuard problem as a three-module campus system covering registered-person recognition, vehicle number-plate reading, and unusual-activity detection, with robustness requirements for lighting, camera angle, and image noise. □filecite□turn0file0□L51-L56□

The report follows the stated CO2–CO5 focus on preprocessing, object detection/recognition, algorithm evaluation, and real-world computer-vision solution design. □filecite□turn0file0□L42-L50□

COMPLETE ASSIGNMENT REQUIREMENTS COVERAGE

This revised report is structured to explicitly cover every deliverable requested for the VisionGuard assignment: problem formulation, objectives, requirements and assumptions, course concepts, methodology, algorithms/pseudocode/flowchart, implementation environment and source-code plan, test cases, execution-output evidence, validation, comparison and trade-offs, SDG relevance, conclusion and limitations, individual contribution, references, and a one-page individual reflection.

Required Component	Report Section
Problem Statement and Problem Formulation	Sections 1–2
Objective and Expected Outcomes	Section 3
Requirements, Constraints and Assumptions	Section 4
Relevant Course Knowledge / Concepts	Section 5
Design / Proposed Solution / Methodology	Sections 6–10
Algorithm / Pseudocode / Flowchart	Section 11
Implementation / Source Code / Environment / Tools	Section 12

Test Cases and Expected/Actual Results	Section 13
Execution Screenshots / Outputs	Section 14
Results and Validation	Section 15
Analysis, Comparison, Trade-offs and Justification	Section 16
SDG / Broader Considerations	Section 17
Conclusion, Limitations and Improvements	Section 18
Individual Contribution	Section 19
References	Section 20
One-Page Individual Reflection	Section 21

1. PROBLEM STATEMENT AND PROBLEM FORMULATION

VisionGuard is proposed as an integrated campus computer-vision system for three security locations: the main entrance, parking entrance, and library. The system must recognize registered students/staff, read vehicle number plates, and flag unusual activity in surveillance video. The assignment specifically requires robustness to lighting variation, camera-angle changes, and image noise.

Problem formulation: Given image or video frames I from campus cameras, the system must transform each frame through preprocessing P , region extraction R , and a module-specific recognition/detection function M to produce a decision D :

$$D = M(R(P(I)))$$

For the gate, D is an identity/access result; for parking, D is a plate string plus confidence; for the library, D is an activity state or alert. The engineering problem is to maximize useful detection/recognition performance while keeping latency, computational cost, training effort, false alarms, and privacy risk within acceptable limits.

1.1 Core problem questions

24. How can noisy and unevenly illuminated CCTV frames be enhanced before analysis?
25. Which traditional and deep-learning techniques are suitable for face recognition, plate reading, and surveillance?
26. How should the final approach be selected using accuracy, adaptability, computation, and deployment constraints?
27. How can the system be evaluated using objective test cases and measurable metrics?
28. How can biometric and vehicle information be handled responsibly?

2. OBJECTIVE AND EXPECTED OUTCOMES

2.1 Objectives

29. Build a modular OpenCV-centered vision pipeline for three campus-security functions.
30. Apply filtering, enhancement, segmentation, morphology, edge/feature extraction, and ROI processing.

31. Compare traditional methods such as Haar/LBPH, template matching, and background subtraction with deep-learning alternatives.
32. Select an appropriate approach for each module and justify the choice.
33. Define an implementation and testing process with measurable performance metrics.
34. Address privacy, security, ethical, and operational requirements.

2.2 Expected outcomes

- A face module capable of returning a registered identity or an unknown/low-confidence result.
- A plate module capable of localizing a plate and producing OCR text with confidence.
- A surveillance module capable of detecting motion/objects and generating candidate unusual-activity alerts.
- A documented comparison between classical and deep-learning approaches.
- A reproducible implementation structure with source code, environment details, testing procedure, and results.
- A responsible deployment design that keeps human review in the loop for consequential decisions.

3. REQUIREMENTS, CONSTRAINTS AND ASSUMPTIONS

3.1 Functional requirements

ID	Requirement
FR1	Capture images/video from gate, parking, and library cameras.
FR2	Preprocess frames using appropriate enhancement and filtering.
FR3	Detect faces and identify registered people.
FR4	Detect number plates and recognize plate characters.
FR5	Detect motion/objects and identify candidate unusual activity.
FR6	Display bounding boxes, labels, confidence and alerts.
FR7	Record necessary event information securely.
FR8	Support testing and performance evaluation.

3.2 Constraints

- Camera resolution, frame rate, angle, and placement may limit performance.
- Low light, glare, blur, occlusion, and noise can reduce recognition quality.
- Deep-learning inference may require greater CPU/GPU resources.
- Training requires representative and appropriately governed datasets.
- Real-time processing creates latency and memory constraints.
- Biometric and vehicle data create privacy and access-control obligations.

3.3 Assumptions

- Cameras are positioned to provide usable views of the intended regions.

- Registered identities and permitted vehicle information are available through an authorized enrollment process.
- The campus can define what constitutes an unusual activity for the surveillance module.
- The implementation will be tested using representative data rather than claiming universal real-world accuracy.
- Model thresholds will be selected using validation data and not the final test set.

4. APPLICATION OF RELEVANT COURSE KNOWLEDGE / CONCEPTS

Concept	Application in VisionGuard
Image acquisition	Read still images and CCTV frames using OpenCV.
Grayscale/color conversion	Reduce representation complexity and prepare images for classical processing/OCR.
Filtering	Gaussian/median filtering for noise reduction.
Contrast enhancement	Histogram equalization/CLAHE and illumination correction.
Segmentation	Thresholding and ROI isolation for plates and motion regions.
Morphology	Opening/closing to clean binary masks.
Edge detection	Locate boundaries and plate/shape structures.
Contours	Identify candidate plate regions and moving objects.
Feature extraction	Classical keypoints or learned face embeddings.
Object detection	Locate vehicles, faces, or relevant objects.
Pattern recognition	Recognize identities and plate character sequences.
Video analysis	Frame differencing/background subtraction/tracking.
Evaluation	Accuracy, precision, recall, F1, OCR accuracy, false-alarm rate and latency.

These concepts directly address the assignment's CO2–CO5 focus on preprocessing, object detection/pattern recognition, algorithm evaluation, and real-world computer-vision solution design.

□ filecite□turn0file0□L42-L50□

5. DESIGN / PROPOSED SOLUTION / METHODOLOGY

The proposed solution uses a hybrid OpenCV-centered architecture. OpenCV provides acquisition, preprocessing, ROI extraction, image operations, video handling and visualization. Deep-learning models are used where robustness is critical, while lightweight classical methods are retained for baselines or efficient screening.

5.1 Face recognition methodology

Frame → face detection → face alignment → normalization → embedding generation → similarity comparison with enrolled identities → threshold check → identity/unknown result.

5.2 Plate recognition methodology

Frame → vehicle/plate detection → plate crop → perspective correction → denoising → contrast enhancement → segmentation → OCR → plate-format validation → text/confidence output.

5.3 Surveillance methodology

Video frame → motion/background analysis → candidate region → object detection/tracking → temporal activity analysis → anomaly rule/model → alert or normal status.

5.4 System flowchart

START → Initialize cameras/models → Capture frame → Preprocess → Identify camera/module → Face / Plate / Surveillance processing → Confidence/validation check → Display result → Log permitted event data → Next frame → STOP when streams are closed.

Flowchart implementation note: the above sequence should be converted into a graphical flowchart for the final submission using standard Start/End, Process, Decision, and Input/Output symbols.

6. ALGORITHM / PSEUDOCODE / FLOWCHART

6.1 Main algorithm

1. Initialize configuration and authorized reference data.
2. Load selected detection, recognition and OCR models.
3. Open the three camera/video sources.
4. Capture a frame and perform quality checks.
5. Preprocess the frame.
6. Route the frame to the corresponding module.
7. Execute detection and recognition/OCR/activity analysis.
8. Apply validated confidence thresholds.
9. Generate a result or human-review flag.
10. Display and securely record the minimum required information.
11. Repeat until the camera stream ends.

6.2 Face pseudocode

FOR each frame: detect faces; FOR each face: align; normalize; generate embedding; compare with enrolled embeddings; IF similarity $\geq$ validated threshold THEN registered identity ELSE unknown; display result.

6.3 Plate pseudocode

Detect vehicle/plate; crop plate; correct perspective; denoise; enhance contrast; segment if useful; run OCR; validate expected plate pattern; return text and confidence.

6.4 Surveillance pseudocode

Compare consecutive frames/background; locate moving regions; detect relevant objects; track across frames; evaluate duration/location/activity conditions; IF validated unusual condition THEN create alert ELSE continue monitoring.

7. IMPLEMENTATION / SOURCE CODE / ENVIRONMENT / TOOLS USED

7.1 Proposed environment

Item	Proposed choice
Programming language	Python
Computer vision	OpenCV
Numerical processing	NumPy
Deep learning	A suitable Python deep-learning framework/model package
Object detection	YOLO-family detector or equivalent
OCR	Suitable OCR engine/model
Development	VS Code / PyCharm / Jupyter as preferred
Version control	Git and GitHub
Operating system	Windows/Linux depending on deployment
Hardware	CPU for classical processing; GPU/accelerator recommended for deep inference

7.2 Source-code organization

project/

├── main.py

├── preprocess.py

├── face_module.py

├── plate_module.py

├── surveillance_module.py

├── evaluation.py

├── visualization.py

└─ data/
└─ models/
└─ results/
└─ README.md

7.3 Source-code evidence

The final submission should place the complete implementation in the GitHub repository required by the assignment. The report should include a short code extract or repository structure, while the full source code should remain in the repository. The assignment explicitly requires source code, pseudocode, dataset information/source, README, installation/execution instructions, results, performance evaluation, and project documentation. ☐ filecite ☐ turn0file0 ☐ L142-L145 ☐

8. TEST CASES AND EXPECTED / ACTUAL RESULTS

Actual results must be filled from the executed implementation. The following test plan provides the required structure without inventing experimental values.

TC ID	Module	Test condition	Expected result	Actual result
TC01	Face	Registered face, good lighting	Correct registered identity	To be measured
TC02	Face	Unknown face	Unknown/rejected result	To be measured
TC03	Face	Side-angle face	Correct identity or low-confidence review	To be measured
TC04	Face	Low-light face	Robust result or safe rejection	To be measured
TC05	Plate	Clear frontal plate	Correct full plate string	To be measured
TC06	Plate	Skewed plate	Correct/acceptable OCR after correction	To be measured
TC07	Plate	Glare/noise	Valid text or low-confidence result	To be measured
TC08	Surveillance	Normal library activity	No false alert	To be measured
TC09	Surveillance	Relevant unusual event	Alert generated	To be measured
TC10	Surveillance	Background movement	Low false-alarm behavior	To be measured

9. EXECUTION SCREENSHOTS / OUTPUTS

The final report must contain screenshots captured from the actual implementation. Since screenshots cannot be truthfully generated without running the user's implementation, the following evidence slots should be replaced with actual outputs.

Figure	Required screenshot	What it should demonstrate
Figure 1	Face detection/recognition output	Face bounding box, identity label, confidence/status
Figure 2	Unknown-face output	Unknown/rejected state
Figure 3	Plate detection output	Vehicle/plate bounding box and detected plate
Figure 4	OCR output	Cropped plate and recognized characters
Figure 5	Surveillance normal frame	Detected motion/objects without unnecessary alert
Figure 6	Surveillance alert frame	Unusual-activity indication and alert status
Figure 7	Application console/UI	Model loading, processing status or result log
Figure 8	GitHub/project structure	Source-code organization and README evidence

Screenshot placement recommendation: place each actual screenshot immediately below its figure caption and add one sentence explaining the observed output. Do not insert fabricated screenshots or claim execution if the program was not run.

10. RESULTS AND VALIDATION

Validation should compare the selected approach with at least one baseline where feasible. Results should be reported separately for face recognition, plate recognition, and surveillance because their error types differ.

Module	Metric	Baseline	Selected method	Observation
Face	Recognition accuracy	To be measured	To be measured	Compare robustness and false recognition
Face	Unknown rejection	To be measured	To be measured	Check safe handling of unregistered faces
Plate	Plate detection precision/recall	To be measured	To be measured	Evaluate localization
Plate	Complete-plate accuracy	To be measured	To be measured	Measures full-string correctness
Plate	Character accuracy	To be measured	To be measured	Measures OCR quality
Surveillance	Precision/recall	To be measured	To be measured	Evaluate event detection

Surveillance	False-alarm rate	To be measured	To be measured	Important for operator usability
All	Inference time	To be measured	To be measured	Assess deployment feasibility

10.1 Validation procedure

35. Keep a separate test set that is not used for training or threshold selection.
36. Run identical test cases through the baseline and selected method.
37. Record predictions, confidence values, processing time, and errors.
38. Compute the chosen metrics.
39. Review representative false positives and false negatives.
40. Repeat difficult conditions such as low light, blur, angle variation, glare, and background movement.
41. Use the results to justify the final selection.

11. ANALYSIS, COMPARISON, TRADE-OFFS AND JUSTIFICATION

Decision factor	Traditional CV	Deep Learning	VisionGuard decision
Accuracy	Strong in controlled scenes	Higher potential in varied scenes	Prefer deep where recognition is critical
Adaptability	Lower	Higher with representative data	Prefer deep for face/plate
Computation	Lower	Higher	Use lightweight models/acceleration where needed
Training	Minimal for classical methods	Required or use pretrained models	Plan validation and model management
Interpretability	High	Lower	Keep classical preprocessing and explicit thresholds
Deployment	Simple	More demanding	Use hybrid architecture
Surveillance efficiency	Fast motion screening	Better object/activity understanding	Hybrid
Maintenance	Rule tuning	Model/data monitoring	Document both

11.1 Final justification

Face recognition: deep-learning embeddings are preferred because identity recognition must handle appearance variation and should not depend solely on handcrafted features.

Plate recognition: deep detection plus OCR is preferred because localization and character recognition must tolerate skew, glare, scale and appearance variation.

Surveillance: a hybrid approach is preferred because simple motion analysis can screen continuous video efficiently, while learned detection/activity analysis can improve interpretation of candidate events.

The assignment explicitly asks for a comprehensive comparison based on accuracy, adaptability, computational cost, training requirements, robustness and deployment suitability; this decision matrix addresses those criteria. □filecite□turn0file0□L79-L88□

12. BROADER CONSIDERATIONS / SDG RELEVANCE

SDG	Connection
SDG 9 – Industry, Innovation and Infrastructure	Applies AI/computer vision to practical security infrastructure.
SDG 11 – Sustainable Cities and Communities	Supports safer, better-managed campus spaces.
SDG 16 – Peace, Justice and Strong Institutions	Supports controlled access and accountable monitoring.

12.1 Privacy and ethics

- Limit collection and retention of biometric and vehicle information.
- Restrict access to authorized personnel.
- Protect stored data using appropriate security controls.
- Provide an unknown/uncertain outcome instead of forcing a recognition decision.
- Use human review for consequential decisions.
- Test for failure modes and operational bias across relevant conditions.
- Document the purpose and authorized scope of monitoring.

13. CONCLUSION, LIMITATIONS AND POSSIBLE IMPROVEMENTS

13.1 Conclusion

VisionGuard demonstrates how image enhancement, segmentation, feature analysis, object detection, recognition, OCR, and video analysis can be integrated into a single campus-security application. The most appropriate solution is not identical for every task. Deep learning is favored for face and plate recognition because adaptability is critical, while surveillance benefits from a hybrid design that combines efficient motion screening with learned object/activity analysis.

13.2 Limitations

- Actual performance depends on camera quality, positioning and deployment conditions.
- Deep models need suitable data and compute resources.
- Anomaly detection is difficult because unusual behavior is context dependent.
- Poor lighting, severe blur and occlusion may still cause errors.
- Thresholds must be validated and monitored over time.
- Privacy requirements can restrict data collection and long-term retention.

13.3 Possible improvements

- Use optimized/accelerated inference for real-time deployment.
- Add multi-camera tracking and event correlation.
- Improve plate perspective correction and OCR confidence fusion.
- Add temporal activity-recognition models.
- Build a dashboard for alerts, camera health and performance.
- Perform periodic validation using appropriately governed deployment data.

14. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Replace the placeholders below with the actual contribution of each team member. Contributions should reflect work genuinely performed.

Member	Role	Contribution
Member 1: _____	Project coordination / implementation	Problem analysis, integration, testing:
Member 2: _____	Face module	Dataset preparation, face detection/recognition, evaluation:
Member 3: _____	Plate module	Plate localization, preprocessing, OCR, evaluation:
Member 4: _____	Surveillance / documentation	Motion/object analysis, visualization, report/GitHub:

If the team has fewer or more members, add/remove rows. Individual contribution should be specific enough to distinguish each member's work.

15. REFERENCES

- **OpenCV Documentation**, *OpenCV – Open Source Computer Vision Library*. OpenCV provides image processing, computer vision, machine-learning algorithms, video processing, object tracking, and related functionality.
- **OpenCV University**, *Curriculum – Fundamentals of Computer Vision and Image Processing with OpenCV*. Useful for image representation, resizing, cropping, contrast enhancement, brightness enhancement, and basic image operations.
- **Ultralytics**, *YOLO Object Detection & Segmentation Documentation*. Used as a reference for YOLO-based object detection, segmentation, tracking, training, validation, prediction, and deployment.
- **Ultralytics**, *Python Usage Documentation*. Reference for integrating YOLO models into Python applications for object detection, segmentation, classification, and related computer-vision tasks.
- **Tesseract OCR**, *Tesseract User Manual*. Reference for optical character recognition and extracting printed text from images, applicable to the vehicle number-plate recognition module.
- **Python Software Foundation**, *Python Documentation*. Reference for the Python programming environment used to implement the VisionGuard application.
- **NumPy Developers**, *NumPy Documentation*. Reference for numerical computation and array-based image processing used with Python computer-vision applications.

- **GitHub**, *GitHub Documentation*. Reference for source-code version control, repository management, README documentation, and project submission.
- **VisionGuard Assignment Brief – ITA05 Computer Vision**, *Multi-Module Vision System Design: Traditional vs. Deep-Learning Approach for Campus Access and Surveillance*. This is the primary source for the problem statement, course outcomes, assessment requirements, deliverables, and SDG mapping.